

TRAJCENTER v2.0 — Protocole de communication PC ↔ Robot via ABB RWS

Version : 2.1 draft
Date : 2026-07-17
Auteurs : J. SCHUMACKER | C. RACINET
RobotWare : 6.x
Transport unique : ABB Robot Web Services
Statut : cahier des charges protocole / routes / encodages
Langage cible : indépendant du langage client
TCP custom : interdit
Polling applicatif des variables RAPID : supprimé
Watchdog filesystem automatique : supprimé

1. Objectif du document

Ce document définit le protocole complet de communication entre un PC applicatif TrajCenter et un contrôleur ABB RobotWare 6.x.

Il décrit :

- les variables RAPID partagées ;
- les routes ABB Robot Web Services utilisées ;
- les abonnements RWS WebSocket ;
- les conventions d'indexation ;
- les formats d'encodage ;
- la pipeline de démarrage ;
- la pipeline de chargement trajectoire ;
- la pipeline de refresh ;
- la pipeline d'annulation ;
- les contraintes de Mastership ;
- les limites dimensionnelles ;
- les cas d'erreur.

Le document est volontairement **indépendant du langage client**.

Un client Python, Node.js, C#, Rust ou autre doit pouvoir implémenter ce protocole sans modifier les routes ni les conventions.

2. Vue d'ensemble

TrajCenter v2.0 utilise exclusivement **ABB Robot Web Services**, abrégé RWS, pour échanger entre le PC et le contrôleur ABB.

Le PC écrit les données trajectoires dans des variables RAPID **PERS**.

Le robot écrit ses demandes opérateur dans des variables RAPID **PERS**.

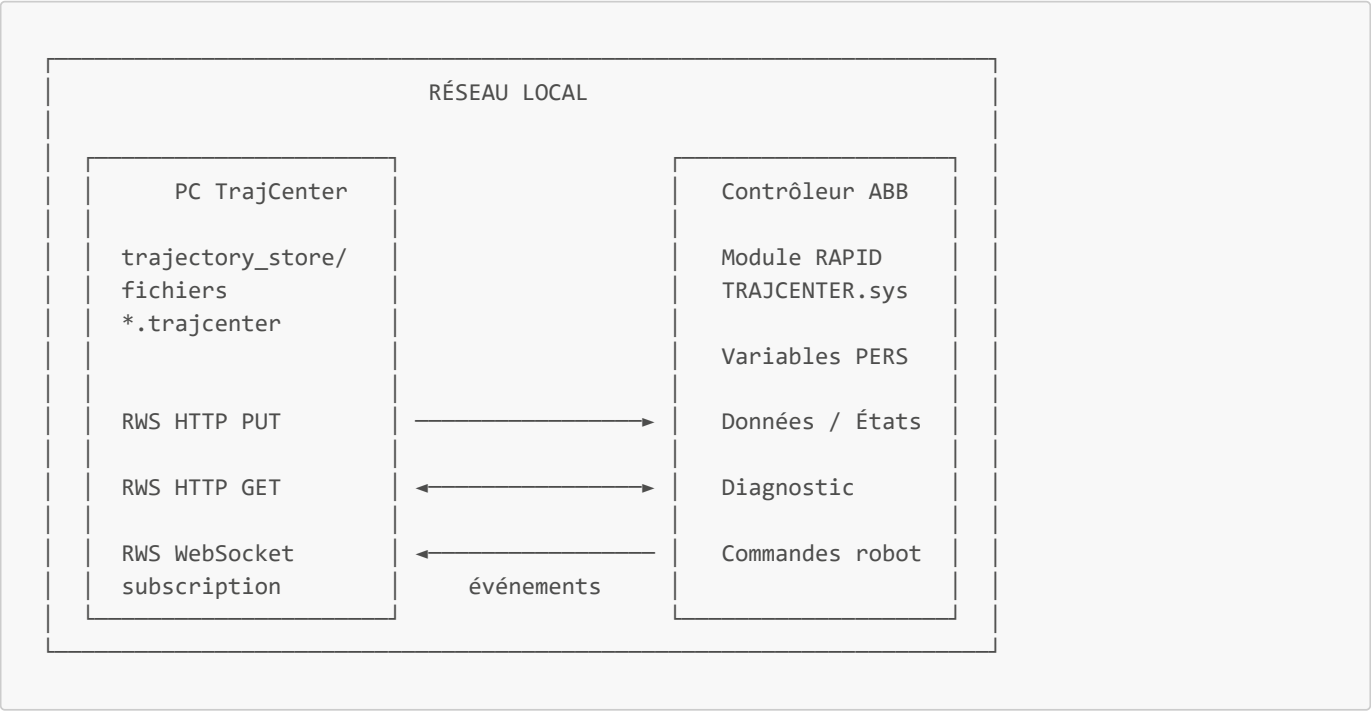
Le PC reçoit les demandes robot via des **abonnements RWS WebSocket**.

Il n'y a plus :

- de serveur TCP custom côté PC ;
- de client TCP RAPID ;
- de port **50000** ;
- d'encodage binaire propriétaire ;

- de paquets TCP de robtargets ;
- de polling périodique de `SelectedTrajIndex` ;
- de watchdog automatique du dossier `trajectory_store/`.

3. Architecture de communication



4. Flux de communication

Flux	Direction	Mécanisme	Rôle
Données trajectoire	PC → Robot	RWS <code>PUT</code>	Robtargets, types de mouvement, vitesses, zones, tools, wobjs
Métadonnées store	PC → Robot	RWS <code>PUT</code>	Liste des trajectoires disponibles
Commandes opérateur	Robot → PC	RWS subscription WebSocket	Demande de chargement, refresh, annulation
États transfert	PC → Robot	RWS <code>PUT</code>	Ready, progress, erreur, transfert en cours
Diagnostic	PC ↔ Robot	RWS <code>GET</code> ponctuel	Resynchronisation, debug, état contrôleur

5. Principes fondamentaux

5.1 Variables RAPID accessibles par RWS

Toutes les variables partagées entre le PC et RAPID doivent être déclarées avec le mot-clé :

```
PERS
```

Les variables non persistantes ou locales ne sont pas utilisées par le protocole.

5.2 Module RAPID système

Les variables partagées sont déclarées dans un module RAPID système :

```
MODULE TRAJCENTER
...
ENDMODULE
```

Le module doit être chargé au démarrage du contrôleur.

5.3 Le PC écrit les données

Le PC est responsable de l'écriture :

- de la liste des trajectoires disponibles ;
 - des données de trajectoire ;
 - des états de transfert ;
 - des messages d'erreur.
-

5.4 Le robot écrit les commandes

Le robot est responsable de l'écriture :

- de l'index de trajectoire sélectionné ;
 - du compteur de demande de chargement ;
 - du compteur de demande de refresh ;
 - du compteur de demande d'annulation.
-

5.5 Événementiel par abonnement RWS

Les demandes robot vers PC sont surveillées par abonnement RWS WebSocket.

Le polling périodique de variables RAPID est interdit pour le fonctionnement nominal.

5.6 Mastership obligatoire pour les écritures RAPID

Toute écriture de variable RAPID par RWS nécessite le Mastership RAPID.

Le Mastership doit être acquis avant une séquence d'écriture et libéré systématiquement après.

6. Constantes du protocole

Les constantes suivantes définissent les dimensions maximales du protocole.

Nom logique	Valeur	Rôle
MAX_TRAJ	50	Nombre maximal de trajectoires listées
MAX_POINTS	100000	Nombre maximal de points dans une trajectoire
MAX_TOOLS	10	Nombre maximal de tools référencés
MAX_WOBSJS	10	Nombre maximal de workobjects référencés

Nom logique	Valeur	Rôle
MAX_SPEEDS	à définir	Nombre maximal de vitesses distinctes
MAX_ZONES	à définir	Nombre maximal de zones distinctes
MAX_MOVE_TYPES	à définir	Nombre maximal de types de mouvement distincts
LAST_ERROR_MAX_LEN	à définir	Longueur maximale du message d'erreur
TRANSFER_BLOCK_SIZE	recommandé 100	Taille logique d'un bloc de transfert
TRAJ_READY_TIMEOUT_S	recommandé 120	Timeout RAPID d'attente de trajectoire
MASTERSHIP_RETRY	recommandé 3	Nombre de tentatives Mastership
SUBSCRIPTION_PRIORITY	recommandé 1	Priorité des ressources RWS
WEBSOCKET_PROTOCOL	robapi2_subscription	Protocole WebSocket ABB RW6

Dans le document, **N** désigne :

$N = \text{MAX_POINTS} = 100000$

Tous les tableaux point-par-point doivent utiliser la même capacité logique **N**.

7. Variables RAPID du protocole

7.1 Métadonnées du store

Ces variables décrivent les trajectoires disponibles dans le dossier local `trajectory_store/`.

Variable	Type RAPID	Taille logique	Écrit par	Lu par	Rôle
NbTrajDispo	num	1	PC	RAPID	Nombre de trajectoires disponibles
NomsTraj{MAX_TRAJ}	string	50	PC	RAPID	Noms des trajectoires affichées
NbPointsTraj{MAX_TRAJ}	num	50	PC	RAPID	Nombre de points par trajectoire

7.2 Commandes robot vers PC

Ces variables sont écrites par RAPID et surveillées par le PC via abonnement RWS.

Variable	Type RAPID	Écrit par	Surveillé par	Rôle
SelectedTrajIndex	num	RAPID	PC	Index de trajectoire sélectionnée
LoadRequestId	num	RAPID	PC	Compteur de demande de chargement
RefreshRequestId	num	RAPID	PC	Compteur de demande de refresh du store
CancelRequestId	num	RAPID	PC	Compteur de demande d'annulation

7.2.1 Pourquoi LoadRequestId est obligatoire

Un abonnement RWS sur `SelectedTrajIndex;value` peut ne produire un événement que si la valeur change réellement.

Pour permettre de recharger la même trajectoire plusieurs fois, le robot doit écrire :

```
SelectedTrajIndex := k;
LoadRequestId := LoadRequestId + 1;
```

Le PC déclenche le chargement sur changement de `LoadRequestId`, puis lit ou utilise la valeur courante de `SelectedTrajIndex`.

7.3 Données trajectoire chargée

Ces variables décrivent la trajectoire actuellement chargée dans le contrôleur.

Variable	Type RAPID	Taille logique	Écrit par	Lu par	Rôle
<code>NbRobttargetsTraj</code>	num	1	PC	RAPID	Nombre de points transférés
<code>RobtTRAJCENTER{MAX_POINTS}</code>	robttarget	100000	PC	RAPID	Robttargets trajectoire
<code>MoveTypeIds{MAX_POINTS}</code>	num	100000	PC	RAPID	Type de mouvement par point
<code>SpeedIndices{MAX_POINTS}</code>	num	100000	PC	RAPID	Index de vitesse par point
<code>ZoneIndices{MAX_POINTS}</code>	num	100000	PC	RAPID	Index de zone par point
<code>ToolIndices{MAX_POINTS}</code>	num	100000	PC	RAPID	Index tool par point
<code>WobjIndices{MAX_POINTS}</code>	num	100000	PC	RAPID	Index wobj par point

7.4 Tables de symboles trajectoire

Ces variables contiennent les noms utilisés par les indices point-par-point.

Variable	Type RAPID	Taille logique	Écrit par	Lu par	Rôle
<code>NbTool</code>	num	1	PC	RAPID	Nombre de tools distincts
<code>ToolNames{MAX_TOOLS}</code>	string	10	PC	RAPID	Noms des tools
<code>NbWobj</code>	num	1	PC	RAPID	Nombre de wobjs distincts
<code>WobjNames{MAX_WOBS}</code>	string	10	PC	RAPID	Noms des wobjs
<code>NbSpeed</code>	num	1	PC	RAPID	Nombre de vitesses distinctes
<code>SpeedNames{MAX_SPEEDS}</code>	string	à définir	PC	RAPID	Noms des vitesses RAPID
<code>NbZone</code>	num	1	PC	RAPID	Nombre de zones distinctes
<code>ZoneNames{MAX_ZONES}</code>	string	à définir	PC	RAPID	Noms des zones RAPID
<code>NbMoveType</code>	num	1	PC	RAPID	Nombre de types de mouvement
<code>MoveTypeNames{MAX_MOVE_TYPES}</code>	string	à définir	PC	RAPID	Noms des types de mouvement

7.5 États PC vers robot

Ces variables sont écrites par le PC et lues par RAPID.

Variable	Type RAPID	Écrit par	Lu par	Rôle
StoreReady	bool	PC	RAPID	Liste trajectoires disponible
TransferInProgress	bool	PC	RAPID	Transfert trajectoire en cours
TrajReady	bool	PC	RAPID	Trajectoire prête à être exécutée
TransferError	bool	PC	RAPID	Erreur pendant le dernier transfert
LastError	string	PC	RAPID	Message d'erreur court
LastLoadedTrajIndex	num	PC	RAPID	Dernier index chargé
LastLoadRequestId	num	PC	RAPID	Dernière demande de chargement traitée
TransferProgress	num	PC	RAPID	Progression transfert, de 0 à 100
TransferCurrentIndex	num	PC	RAPID	Dernier point transféré

8. Convention d'indexation

8.1 Indexation des tableaux RAPID

Les tableaux RAPID sont adressés avec des accolades :

```
RobtTRAJCENTER{1}
NomsTraj{1}
ToolNames{1}
```

En URL RWS, les accolades doivent être encodées :

```
RobtTRAJCENTER%7B1%7D
NomsTraj%7B1%7D
ToolNames%7B1%7D
```

8.2 Index trajectoires

Les trajectoires affichées à l'opérateur sont indexées côté RAPID selon la convention menu retenue.

Convention recommandée :

```
SelectedTrajIndex = 1..NbTrajDispo
SelectedTrajIndex = 0 signifie aucune sélection
```

8.3 Indices point-par-point pour tools, wobjs, speeds et zones

Décision actuelle :

Les indices transférés dans les tableaux point-par-point conservent l'indexation du fichier .trajcenter.

Donc :

```
tool_index fichier = 0 -> ToolIndices{i} = 0
wobj_index fichier = 0 -> WobjIndices{i} = 0
```

Même principe pour les vitesses, zones et types de mouvement si une table de symboles est utilisée en base 0.

Attention : cette convention diffère de l'adressage des tableaux RAPID eux-mêmes.

L'élément RAPID reste adressé en base 1 :

```
ToolIndices{1} := 0;
WobjIndices{1} := 0;
```

Ici :

- `{1}` est l'index du point RAPID ;
- `0` est l'index logique du tool ou du wobj dans le fichier / table métier.

9. Format `.trajcenter`

Un fichier `.trajcenter` représente une trajectoire complète.

Le format interne recommandé est une archive contenant au minimum :

```
meta.json
points.parquet
tools.json
wobjs.json
```

Selon les besoins, il peut aussi contenir :

```
speeds.json
zones.json
move_types.json
```

9.1 Colonnes minimales de `points.parquet`

Colonne	Type logique	Rôle
<code>x</code>	float	Position X en mm
<code>y</code>	float	Position Y en mm
<code>z</code>	float	Position Z en mm
<code>q1</code>	float	Quaternion ABB composante scalaire
<code>q2</code>	float	Quaternion ABB composante X
<code>q3</code>	float	Quaternion ABB composante Y
<code>q4</code>	float	Quaternion ABB composante Z

Colonne	Type logique	Rôle
<code>cf1</code>	num	Configuration robot axe 1
<code>cf4</code>	num	Configuration robot axe 4
<code>cf6</code>	num	Configuration robot axe 6
<code>cfx</code>	num	Configuration robot étendue
<code>move_type</code>	string ou index	Type de mouvement
<code>speed</code>	string ou index	Vitesse
<code>zone</code>	string ou index	Zone
<code>tool_index</code>	num	Index tool
<code>wobj_index</code>	num	Index wobj

9.2 Colonnes optionnelles axes externes

Colonne	Type logique	Rôle
<code>eax_a</code>	float	Axe externe A
<code>eax_b</code>	float	Axe externe B
<code>eax_c</code>	float	Axe externe C
<code>eax_d</code>	float	Axe externe D
<code>eax_e</code>	float	Axe externe E
<code>eax_f</code>	float	Axe externe F

Une colonne d'axe externe absente signifie :

```
axe externe inactif
```

La valeur ABB d'axe externe inactif est injectée uniquement lors de la sérialisation RWS.

La valeur d'axe externe inactif ne doit pas être stockée dans le fichier `.trajcenter`.

10. Format RWS d'un `robtarget`

10.1 Structure

Un `robtarget` ABB contient 17 valeurs numériques organisées en quatre groupes :

```
[[x,y,z],[q1,q2,q3,q4],[cf1,cf4,cf6,cfx],[eax_a,eax_b,eax_c,eax_d,eax_e,eax_f]]
```

10.2 Exemple

```
[[1500,0,1789],[0,0,1,0],[0,0,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]]
```


10.3 Quaternion ABB

ABB utilise la convention quaternion scalaire en premier :

$$[q1, q2, q3, q4] = [w, x, y, z]$$

Cette convention doit être respectée dans tous les convertisseurs.

10.4 Axes externes inactifs

Un axe externe inactif est sérialisé avec :

9E+9

Cette valeur est une convention RWS/RAPID.

Elle ne doit pas être stockée dans le fichier `.trajcenter`.

11. Encodage des informations point-par-point

Deux options d'encodage sont décrites dans ce document.

La décision définitive sera prise après validation projet.

11.1 Option A — Encodage direct par chaînes point-par-point

Dans cette option, chaque point contient directement les chaînes nécessaires.

Variables possibles :

```
PERS string MoveTypes{MAX_POINTS};
PERS string SpeedNamesPerPoint{MAX_POINTS};
PERS string ZoneNamesPerPoint{MAX_POINTS};
PERS num ToolIndices{MAX_POINTS};
PERS num WobjIndices{MAX_POINTS};
```

Exemple logique :

```
RobtTRAJCENTER{1}      = [[...]]
MoveTypes{1}            = "MoveL"
SpeedNamesPerPoint{1}   = "v500"
ZoneNamesPerPoint{1}    = "z10"
ToolIndices{1}          = 0
WobjIndices{1}          = 0
```

Avantages :

- très lisible ;
- proche du contenu du fichier `.trajcenter` ;
- simple à produire côté client.

Inconvénients :

- mémoire importante pour `MAX_POINTS = 100000` ;
- traitement RAPID plus lourd ;
- mapping string vers données RAPID nécessaires ;
- moins performant pour les très grandes trajectoires.

11.2 Option B — Tables de symboles et indices point-par-point

Dans cette option, les noms sont transférés une seule fois dans des tables de symboles, puis chaque point référence ces tables par indices numériques.

Variables possibles :

```
PERS num MoveTypeIds{MAX_POINTS};
PERS num SpeedIndices{MAX_POINTS};
PERS num ZoneIndices{MAX_POINTS};
PERS num ToolIndices{MAX_POINTS};
PERS num WobjIndices{MAX_POINTS};

PERS num NbMoveType;
PERS string MoveTypeNames{MAX_MOVE_TYPES};

PERS num NbSpeed;
PERS string SpeedNames{MAX_SPEEDS};

PERS num NbZone;
PERS string ZoneNames{MAX_ZONES};

PERS num NbTool;
PERS string ToolNames{MAX_TOOLS};

PERS num NbWobj;
PERS string WobjNames{MAX_WOBS};
```

Exemple logique :

```
MoveTypeNames{1} = "MoveL"
SpeedNames{1}    = "v500"
ZoneNames{1}     = "z10"
ToolNames{1}     = "tool0"
WobjNames{1}     = "wobj0"

RobtTRAJCENTER{1} = [[...]]
MoveTypeIds{1}    = 0
SpeedIndices{1}   = 0
ZoneIndices{1}    = 0
ToolIndices{1}    = 0
WobjIndices{1}    = 0
```

Dans cet exemple, les valeurs 0 sont des indices métier issus du fichier `.trajcenter`.

Avantages :

- beaucoup moins de mémoire que les chaînes point-par-point ;
- plus stable pour 100000 points ;

- plus facile à valider ;
- plus adapté à une exécution RAPID déterministe ;
- protocole mieux normalisé.

Inconvénients :

- nécessite une étape de mapping côté client ;
- nécessite une logique de résolution côté RAPID.

11.3 Option recommandée

L'option recommandée pour une trajectoire pouvant atteindre 100000 points est :

```
Option B – Tables de symboles et indices point-par-point
```

L'option A reste documentée temporairement pour arbitrage projet.

12. Routes RWS générales

12.1 Base URL

Toutes les routes RWS sont relatives à :

```
http://<ROBOT_IP>
```

Exemple :

```
http://192.168.125.1/rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex
```

12.2 Authentification

Les requêtes RWS utilisent :

```
HTTP Digest Authentication
```

Credentials par défaut RobotWare :

```
User: Default User  
Password: robotics
```

Le client doit conserver la session et les cookies RWS, notamment :

```
ABBCX
```

12.3 Format des données

Les écritures de variables RAPID utilisent des corps de type formulaire :

```
value=<VALEUR_RAPID>
```

Le format exact dépend du client HTTP, mais le contenu logique doit être équivalent à :

```
Content-Type: application/x-www-form-urlencoded
```

12.4 Route générique de lecture d'un symbole RAPID

```
GET /rw/rapid/symbol/data/<SYMBOL_URL>
```

Avec :

```
<SYMBOL_URL> = RAPID/<TASK>/<MODULE>/<VARIABLE>
```

Exemple :

```
GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex
```

12.5 Route générique d'écriture d'un symbole RAPID

```
PUT /rw/rapid/symbol/data/<SYMBOL_URL>
```

Exemple :

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady  
value=TRUE
```

12.6 Écriture d'un élément de tableau RAPID

En RAPID :

```
RobtTRAJCENTER{1}
```

En URL RWS :

```
RobtTRAJCENTER%7B1%7D
```

Route complète :

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER%7B1%7D
```

13. Routes de lecture ponctuelle

Les lectures ponctuelles sont utilisées pour :

- diagnostic ;
- démarrage ;
- reconnexion ;
- resynchronisation ;
- validation.

Elles ne doivent pas remplacer les abonnements RWS dans le fonctionnement nominal.

ID	Route	Rôle
G1	GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex	Lire l'index sélectionné
G2	GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LoadRequestId	Lire la dernière demande de chargement
G3	GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RefreshRequestId	Lire la dernière demande de refresh
G4	GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/CancelRequestId	Lire la dernière demande d'annulation
G5	GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTrajDispo	Lire le nombre de trajectoires disponibles
G6	GET /rw/rapid/execution	Lire l'état d'exécution RAPID

14. Routes d'écriture des métadonnées store

Toutes les écritures nécessitent Mastership.

ID	Route	Valeur	Rôle
W_STORE_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/StoreReady	FALSE	Début mise à jour store
W_STORE_2	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTrajDispo	N	Nombre de trajectoires
W_STORE_3_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NomsTraj%7Bi%7D	"name"	Nom trajectoire <i>i</i>
W_STORE_4_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbPointsTraj%7Bi%7D	count	Nombre de points trajectoire <i>i</i>

ID	Route	Valeur	Rôle
W_STORE_5	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/StoreReady	TRUE	Store disponible

Note :

```
i = 1..MAX_TRAJ pour l'adressage tableau RAPID
```

Les entrées non utilisées doivent être vidées ou laissées dans un état défini selon la convention retenue.

Convention recommandée :

```
Pour i > NbTrajDispo :
NomsTraj{i} = ""
NbPointsTraj{i} = 0
```

15. Routes d'écriture des données trajectoire

Toutes les écritures nécessitent Mastership.

15.1 États de début de transfert

ID	Route	Valeur
W_TR_START_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferInProgress	TRUE
W_TR_START_2	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady	FALSE
W_TR_START_3	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferError	FALSE
W_TR_START_4	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastError	""
W_TR_START_5	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferProgress	0
W_TR_START_6	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferCurrentIndex	0

15.2 Taille trajectoire

ID	Route	Valeur
W_TR_SIZE_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbRobtargetsTraj	nombre de points

15.3 Tables tools et wobj

ID	Route	Valeur
W_TOOL_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTool	nombre de tools
W_TOOL_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ToolNames%7Bi%7D	"tool_name"
W_WOBJ_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbWobj	nombre de wobjs
W_WOBJ_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/WobjNames%7Bi%7D	"wobj_name"

15.4 Tables speeds, zones et move types

Si l'option B d'encodage est retenue :

ID	Route	Valeur
W_SPEED_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbSpeed	nombre de vitesses
W_SPEED_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SpeedNames%7Bi%7D	"v500"
W_ZONE_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbZone	nombre de zones
W_ZONE_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ZoneNames%7Bi%7D	"z10"
W_MOVE_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbMoveType	nombre de types
W_MOVE_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/MoveTypeNames%7Bi%7D	"MoveL"

15.5 Données point par point — option B recommandée

Pour chaque point *i*, avec *i* = 1..*NbRobttargetsTraj* pour l'adressage RAPID :

ID	Route	Valeur
W_POINT_ROBT_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER%7Bi%7D	robttarget RWS
W_POINT_MOVE_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/MoveTypeIds%7Bi%7D	index type mouvement
W_POINT_SPEED_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SpeedIndices%7Bi%7D	index vitesse
W_POINT_ZONE_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ZoneIndices%7Bi%7D	index zone
W_POINT_TOOL_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ToolIndices%7Bi%7D	index tool
W_POINT_WOBJ_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/WobjIndices%7Bi%7D	index wobj

Exemple :

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER%7B1%7D
value=[[1500,0,1789],[0,0,1,0],[0,0,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]]
```

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/MoveTypeIds%7B1%7D
value=0
```

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SpeedIndices%7B1%7D
value=0
```

15.6 Données point par point — option A alternative

Si l'option A est retenue temporairement ou définitivement :

ID	Route	Valeur
W_POINT_ROBT_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER%7Bi%7D	robttarget RWS
W_POINT_MOVE_STR_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/MoveTypes%7Bi%7D	"MoveL"
W_POINT_SPEED_STR_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SpeedNamesPerPoint%7Bi%7D	"v500"
W_POINT_ZONE_STR_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ZoneNamesPerPoint%7Bi%7D	"z10"
W_POINT_TOOL_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ToolIndices%7Bi%7D	index tool
W_POINT_WOBJ_i	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/WobjIndices%7Bi%7D	index wobj

15.7 États de fin de transfert nominal

L'ordre de fin est important.

TrajReady doit être écrit uniquement lorsque toutes les données nécessaires ont été écrites.

ID	Route	Valeur
W_TR_END_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastLoadedTrajIndex	index chargé
W_TR_END_2	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastLoadRequestId	request id traité
W_TR_END_3	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferProgress	100
W_TR_END_4	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferInProgress	FALSE
W_TR_END_5	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady	TRUE

15.8 États de fin en erreur

En cas d'erreur pendant le transfert :

ID	Route	Valeur
W_TR_ERR_1	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferError	TRUE
W_TR_ERR_2	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastError	message court
W_TR_ERR_3	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferInProgress	FALSE
W_TR_ERR_4	PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady	FALSE

16. Mastership

16.1 Routes Mastership

ID	Route	Rôle
M1	POST /rw/mastership/request	Acquisition Mastership
M2	POST /rw/mastership/release	Libération Mastership

16.2 Règles

Toute écriture de variable RAPID via RWS doit être faite sous Mastership.

Le Mastership doit être libéré :

- après succès ;
- après erreur ;
- après annulation ;
- après timeout ;
- après interruption du client si possible.

Le client doit utiliser une structure équivalente à :

```
request mastership
try:
    write variables
finally:
    release mastership
```

16.3 Contraintes RobotWare

Le Mastership peut être refusé si :

- un autre client possède déjà le Mastership ;
- le contrôleur est dans un état incompatible ;
- le programme RAPID est en cours d'exécution selon le mode et la configuration ;
- l'utilisateur RWS n'a pas les droits suffisants.

Le client doit prévoir un nombre borné de retries.

17. Abonnements RWS

17.1 Principe

Le PC crée un abonnement RWS aux variables de commande RAPID.

Route ABB :

```
POST /subscription
```

Le contrôleur retourne une URL WebSocket :

```
ws://<ROBOT_IP>:80/poll/<SUBSCRIPTION_ID>
```

Le client ouvre la WebSocket avec le sous-protocole :

```
robapi2_subscription
```

17.2 Variables à surveiller

Le PC doit surveiller :

```
SelectedTrajIndex  
LoadRequestId  
RefreshRequestId  
CancelRequestId
```

La variable principale de déclenchement du chargement est :

```
LoadRequestId
```

La variable `SelectedTrajIndex` donne le contenu de la demande.

17.3 Création de subscription

Exemple de corps de requête :

```
resources=4  
1=/rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex;value  
1-p=1  
2=/rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LoadRequestId;value  
2-p=1  
3=/rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RefreshRequestId;value  
3-p=1  
4=/rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/CancelRequestId;value  
4-p=1
```

17.4 Connexion WebSocket

Le client doit se connecter à l'URL fournie par le contrôleur.

Header obligatoire :

```
Sec-WebSocket-Protocol: robapi2_subscription
```

Ne pas utiliser :

```
rws_subscription
```

car certains contrôleurs RobotWare 6 le refusent.

17.5 Suppression de subscription

À l'arrêt propre du client :

```
DELETE /subscription/<SUBSCRIPTION_ID>
```

17.6 Reconnexion

En cas de coupure WebSocket, le client doit :

1. Fermer la WebSocket locale.
2. Supprimer la subscription si possible.
3. Recréer une subscription.
4. Reconnecter la WebSocket.
5. Relire ponctuellement :
 - SelectedTrajIndex
 - LoadRequestId
 - RefreshRequestId
 - CancelRequestId
6. Comparer avec les dernières valeurs traitées.
7. Traiter toute demande non encore acquittée.

18. Pipeline de démarrage PC

Au lancement du client TrajCenter :

1. Ouvrir une session RWS.
2. Vérifier la disponibilité du contrôleur.
3. Créer l'abonnement RWS aux commandes :
 - SelectedTrajIndex
 - LoadRequestId
 - RefreshRequestId
 - CancelRequestId
4. Ouvrir la WebSocket de subscription.
5. Scanner une première fois trajectory_store/.
6. Lire les fichiers .trajcenter valides.
7. Acquérir le Mastership.
8. StoreReady := FALSE.
9. Écrire NbTrajDispo.
10. Écrire NomsTraj{i}.
11. Écrire NbPointsTraj{i}.
12. StoreReady := TRUE.
13. Libérer le Mastership.
14. Entrer dans la boucle événementielle WebSocket.

19. Pipeline de chargement trajectoire

19.1 Côté RAPID

Lorsqu'un opérateur sélectionne une trajectoire **k** :

```
SelectedTrajIndex := k;  
LoadRequestId := LoadRequestId + 1;
```

Puis RAPID attend :

```
WaitUntil TrajReady = TRUE \MaxTime:=120;
```

ou surveille :

```
TransferError
```

19.2 Côté PC

À réception d'un événement sur `LoadRequestId` :

1. Lire la nouvelle valeur de `LoadRequestId`.
2. Lire ou utiliser la valeur courante de `SelectedTrajIndex`.
3. Vérifier que `LoadRequestId` n'a pas déjà été traité.
4. Vérifier que `SelectedTrajIndex` est valide.
5. Charger le fichier `.trajcenter` correspondant.
6. Valider le fichier.
7. Vérifier les limites :
 - `points <= MAX_POINTS`
 - `trajectoires <= MAX_TRAJ`
 - `tools <= MAX_TOOLS`
 - `wobjs <= MAX_WOBS`
 - `speeds <= MAX_SPEEDS`
 - `zones <= MAX_ZONES`
8. Acquérir le Mastership.
9. Écrire les états de début :
 - `TransferInProgress := TRUE`
 - `TrajReady := FALSE`
 - `TransferError := FALSE`
 - `LastError := ""`
 - `TransferProgress := 0`
 - `TransferCurrentIndex := 0`
10. Écrire `NbRobtargetsTraj`.
11. Écrire les tables de symboles.
12. Écrire les données point par point.
13. Mettre à jour la progression par blocs.
14. Écrire les états de fin :
 - `LastLoadedTrajIndex`
 - `LastLoadRequestId`
 - `TransferProgress := 100`
 - `TransferInProgress := FALSE`
 - `TrajReady := TRUE`
15. Libérer le Mastership.

20. Transfert logique par blocs

Le transfert RWS reste constitué d'écritures élémentaires, mais le client doit organiser le transfert en blocs logiques.

Exemple avec :

```
TRANSFER_BLOCK_SIZE = 100
```

Pipeline :

```
Bloc 1 : points 1..100  
Bloc 2 : points 101..200  
Bloc 3 : points 201..300  
...
```

Après chaque bloc, le PC écrit :

```
TransferCurrentIndex = dernier point écrit  
TransferProgress = pourcentage entier
```

Exemple :

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferCurrentIndex  
value=500
```

```
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferProgress  
value=25
```

Le client doit vérifier régulièrement si une annulation a été demandée.

21. Pipeline d'annulation

21.1 Côté RAPID

Pour demander une annulation :

```
CancelRequestId := CancelRequestId + 1;
```

21.2 Côté PC

À réception d'un événement sur `CancelRequestId` :

1. Mémoriser la nouvelle demande d'annulation.
2. Si aucun transfert n'est en cours, ne rien interrompre.
3. Si un transfert est en cours, arrêter après le point courant ou le bloc courant.
4. Écrire :
 - TransferInProgress := FALSE
 - TransferError := TRUE
 - LastError := "Transfer cancelled"

```
- TrajReady := FALSE
5. Libérer le Mastership si détenu.
```

22. Pipeline de refresh sans watchdog

22.1 Côté RAPID

Pour demander un refresh du store :

```
RefreshRequestId := RefreshRequestId + 1;
```

22.2 Côté PC

À réception d'un événement sur `RefreshRequestId` :

```
1. Lire la nouvelle valeur de RefreshRequestId.
2. Vérifier qu'elle n'a pas déjà été traitée.
3. Scanner trajectory_store/.
4. Lire les fichiers .trajcenter valides.
5. Acquérir le Mastership.
6. StoreReady := FALSE.
7. Réécrire NbTrajDispo.
8. Réécrire NomsTraj{i}.
9. Réécrire NbPointsTraj{i}.
10. Nettoyer les entrées non utilisées.
11. StoreReady := TRUE.
12. Libérer le Mastership.
```

Le refresh ne nécessite pas de redémarrer le client PC.

23. Pipeline d'exécution RAPID

Après réception de `TrajReady = TRUE`, RAPID peut exécuter la trajectoire.

Pseudo-logique :

```
IF TransferError THEN
  ! Afficher LastError
ELSE
  WaitUntil TrajReady = TRUE \MaxTime:=120;

  IF TrajReady THEN
    TrajReady := FALSE;

    FOR i FROM 1 TO NbRobtargetsTraj DO
      ! Résoudre MoveTypeIds{i}
      ! Résoudre SpeedIndices{i}
      ! Résoudre ZoneIndices{i}
      ! Résoudre ToolIndices{i}
      ! Résoudre WobjIndices{i}
      ! Exécuter MoveL / MoveJ / autre
    ENDFOR
```

```
ENDIF  
ENDIF
```

La résolution exacte des indices vers les données RAPID déclarées dépend du module RAPID d'exécution.

24. Encodage URL

24.1 Variables simples

```
SelectedTrajIndex  
LoadRequestId  
TrajReady
```

Aucun encodage spécial nécessaire.

24.2 Tableaux RAPID

RAPID :

```
Variable{i}
```

URL :

```
Variable%7Bi%7D
```

Exemples :

```
RobtTRAJCENTER{1} -> RobtTRAJCENTER%7B1%7D  
NomsTraj{3}       -> NomsTraj%7B3%7D  
ToolNames{2}      -> ToolNames%7B2%7D
```

25. Encodage des valeurs RAPID

25.1 num

```
value=42  
value=3.14  
value=0
```

25.2 bool

```
value=TRUE  
value=FALSE
```

25.3 string

Les chaînes doivent être encodées comme chaînes RAPID.

Exemple logique :

```
value="traj_001"
```

Le client HTTP doit gérer l'encodage formulaire correctement.

25.4 robtarget

Les robtargets doivent être encodés comme données RAPID complexes non quotées.

Exemple :

```
value=[[1500,0,1789],[0,0,1,0],[0,0,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]]
```

Ne pas envoyer :

```
value="[[1500,0,1789],...]"
```

Ne pas envoyer de JSON objet.

26. États et invariants

26.1 Invariant de transfert nominal

Pendant transfert :

```
TransferInProgress = TRUE  
TrajReady = FALSE  
TransferError = FALSE
```

Après transfert réussi :

```
TransferInProgress = FALSE  
TrajReady = TRUE  
TransferError = FALSE  
TransferProgress = 100  
LastLoadedTrajIndex = SelectedTrajIndex traité  
LastLoadRequestId = LoadRequestId traité
```

Après erreur :

```
TransferInProgress = FALSE  
TrajReady = FALSE
```



```
TransferError = TRUE
LastError != ""
```

26.2 Invariant store

Pendant refresh :

```
StoreReady = FALSE
```

Après refresh réussi :

```
StoreReady = TRUE
NbTrajDispo = nombre de trajectoires valides
```

27. Gestion des erreurs

Situation	Comportement PC	Comportement RAPID attendu
Mastership refusé	Retry borné puis erreur	Afficher ou attendre
Fichier <code>.trajcenter</code> absent	<code>TransferError := TRUE</code>	Afficher <code>LastError</code>
Fichier corrompu	<code>TransferError := TRUE</code>	Afficher <code>LastError</code>
Index hors bornes	Refuser chargement	Afficher erreur
Trop de points	Refuser chargement	Afficher erreur
Trop de tools	Refuser chargement	Afficher erreur
Trop de wobjs	Refuser chargement	Afficher erreur
Trop de speeds	Refuser chargement	Afficher erreur
Trop de zones	Refuser chargement	Afficher erreur
Perte réseau pendant transfert	Arrêt, release si possible	Timeout attente
Perte WebSocket	Reconnexion et resync	Aucun comportement spécial
Annulation opérateur	Stop transfert, erreur contrôlée	Retour menu
Timeout RAPID <code>TrajReady</code>	Aucun ou diagnostic	Retour menu / erreur

28. Resynchronisation

Après reconnexion ou redémarrage du client PC, le client doit relire :

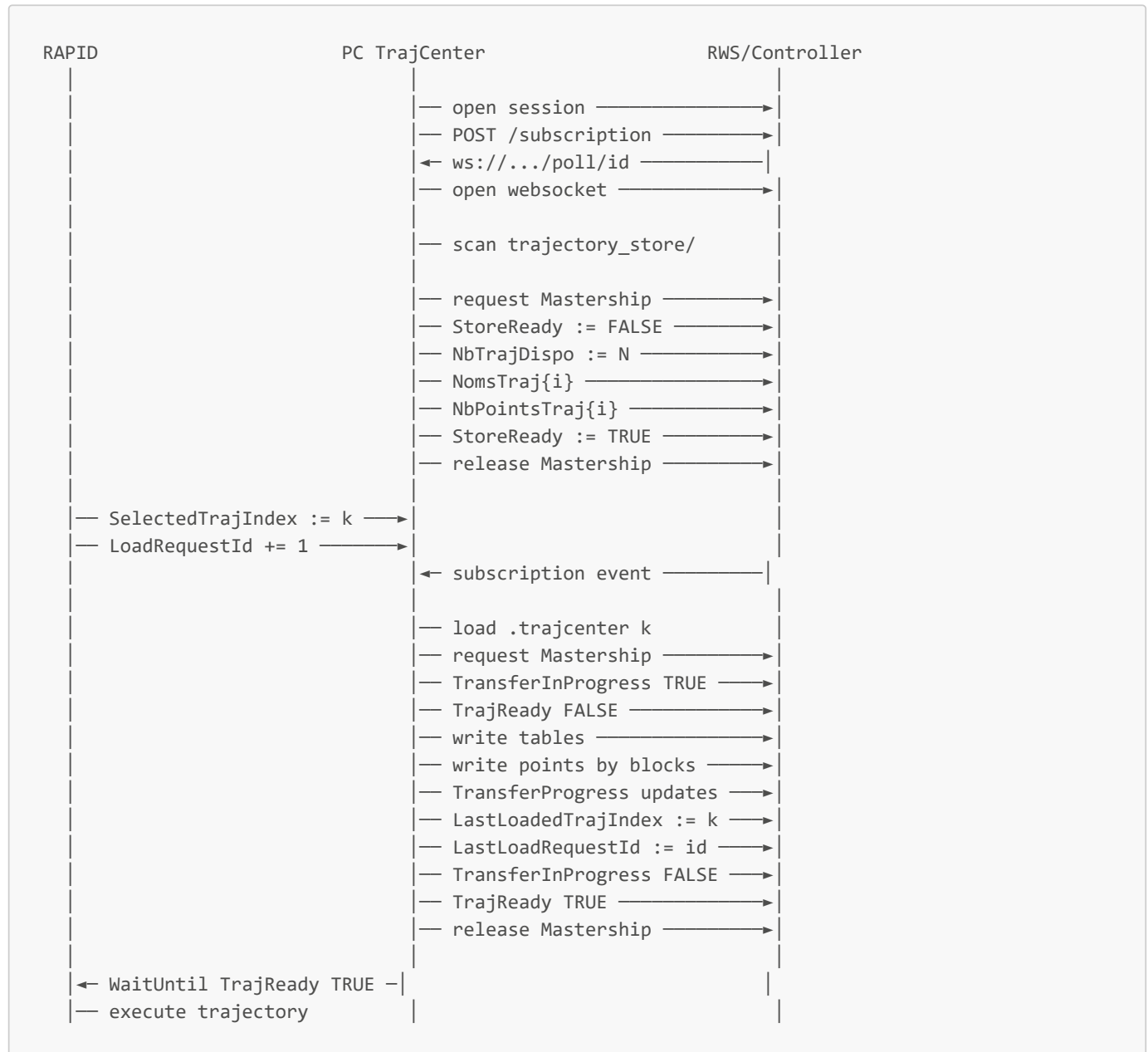
```
SelectedTrajIndex
LoadRequestId
RefreshRequestId
CancelRequestId
LastLoadRequestId
```

Si :

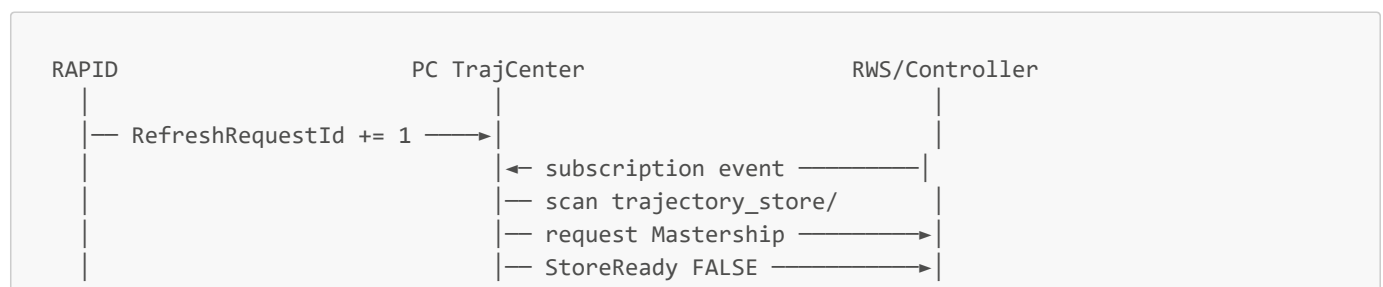
LoadRequestId > LastLoadRequestId

alors le client doit considérer qu'une demande de chargement est en attente et la traiter.

29. Séquence complète nominale



30. Séquence refresh

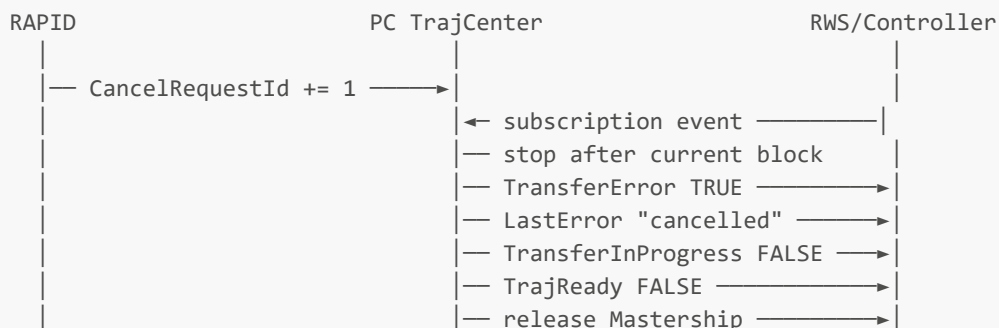


```

|— update metadata —————>|
|— StoreReady TRUE —————>|
|— release Mastership —————>|

```

31. Séquence annulation



32. Estimation mémoire

Un **robtarget** contient 17 valeurs numériques.

Mémoire brute approximative pour les robtargets :

$$\text{Mémoire robtargets}(N) = N \times 17 \times 4 \text{ octets}$$

Pour $N = 100000$:

$$100000 \times 17 \times 4 = 6800000 \text{ octets}$$

Soit environ :

6.8 MB bruts
environ 7.8 MB avec overhead estimé

Les tableaux parallèles ajoutent une mémoire significative, en particulier si l'option A avec chaînes point-par-point est retenue.

L'option B avec indices numériques est recommandée pour limiter la mémoire.

33. Limites et validations obligatoires

Avant transfert, le client doit valider :

```

NbTrajDispo <= MAX_TRAJ
NbRobtargetsTraj <= MAX_POINTS
NbTool <= MAX_TOOLS
NbWobj <= MAX_WOBS
NbSpeed <= MAX_SPEEDS
NbZone <= MAX_ZONES
NbMoveType <= MAX_MOVE_TYPES

```

Le client doit refuser tout transfert si une limite est dépassée.

34. Checklist de conformité client

Un client conforme doit :

- utiliser uniquement ABB RWS ;
- ne pas ouvrir de serveur TCP custom ;
- ne pas utiliser de polling nominal de `SelectedTrajIndex` ;
- créer une subscription RWS ;
- utiliser `robapi2_subscription` comme sous-protocole WebSocket ;
- écrire les variables RAPID uniquement sous Mastership ;
- libérer le Mastership en cas d'erreur ;
- encoder les tableaux RAPID avec `{i}` et `%7B i %7D` en URL ;
- encoder les robtargets en format RAPID non quoté ;
- injecter `9E+9` uniquement à la sérialisation RWS ;
- ne pas stocker `9E+9` dans les fichiers `.trajcenter` ;
- gérer le rechargement de la même trajectoire via `LoadRequestId` ;
- gérer le refresh via `RefreshRequestId` ;
- gérer l'annulation via `CancelRequestId` ;
- mettre `TrajReady` à `TRUE` uniquement en toute fin de transfert ;
- mettre `TransferError` à `TRUE` en cas d'échec ;
- mettre à jour `LastLoadRequestId` après traitement nominal ;
- resynchroniser les compteurs après reconnexion.

35. Points à arbitrer

Les points suivants restent à valider projet :

35.1 Encodage point-par-point

Deux options sont temporairement documentées :

```
Option A : chaînes point-par-point
Option B : tables de symboles + indices
```

Option recommandée :

```
Option B
```

35.2 Taille des tables de symboles

À définir :

```
MAX_SPEEDS
MAX_ZONES
MAX_MOVE_TYPES
LAST_ERROR_MAX_LEN
```

35.3 Convention d'indices métier

Décision actuelle :

Les indices métier transférés dans ToolIndices, WobjIndices, SpeedIndices, ZoneIndices et MoveTypeIds conservent la base du fichier .trajcenter.

Donc typiquement :

0 = premier élément métier
1 = deuxième élément métier

L'adressage des tableaux RAPID reste en base 1.

36. Résumé des variables recommandées

```
! Store metadata
PERS num NbTrajDispo;
PERS string NomsTraj{50};
PERS num NbPointsTraj{50};

! Robot -> PC commands
PERS num SelectedTrajIndex;
PERS num LoadRequestId;
PERS num RefreshRequestId;
PERS num CancelRequestId;

! Trajectory data
PERS num NbRobtargetsTraj;
PERS robtarjet RobtTRAJCENTER{100000};

! Option B point metadata
PERS num MoveTypeIds{100000};
PERS num SpeedIndices{100000};
PERS num ZoneIndices{100000};
PERS num ToolIndices{100000};
PERS num WobjIndices{100000};

! Symbol tables
PERS num NbTool;
PERS string ToolNames{10};

PERS num NbWobj;
PERS string WobjNames{10};

PERS num NbSpeed;
PERS string SpeedNames{MAX_SPEEDS};

PERS num NbZone;
PERS string ZoneNames{MAX_ZONES};

PERS num NbMoveType;
PERS string MoveTypeNames{MAX_MOVE_TYPES};

! PC -> Robot states
PERS bool StoreReady;
PERS bool TransferInProgress;
```

```
PERS bool TrajReady;  
PERS bool TransferError;  
PERS string LastError;  
PERS num LastLoadedTrajIndex;  
PERS num LastLoadRequestId;  
PERS num TransferProgress;  
PERS num TransferCurrentIndex;
```

37. Résumé des routes principales

```
POST /subscription  
DELETE /subscription/<SUBSCRIPTION_ID>  
  
POST /rw/mastership/request  
POST /rw/mastership/release  
  
GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SelectedTrajIndex  
GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LoadRequestId  
GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RefreshRequestId  
GET /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/CancelRequestId  
  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbTrajDispo  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NomsTraj%7Bi%7D  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbPointsTraj%7Bi%7D  
  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/NbRobtargetsTraj  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/RobtTRAJCENTER%7Bi%7D  
  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/MoveTypeIds%7Bi%7D  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/SpeedIndices%7Bi%7D  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ZoneIndices%7Bi%7D  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/ToolIndices%7Bi%7D  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/WobjIndices%7Bi%7D  
  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/StoreReady  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferInProgress  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TrajReady  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferError  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastError  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastLoadedTrajIndex  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/LastLoadRequestId  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferProgress  
PUT /rw/rapid/symbol/data/RAPID/T_ROB1/TRAJCENTER/TransferCurrentIndex
```